Data Preparation Essentials

Section 1: Learn

What is Data Preparation?

Data preparation is the process of **cleaning, transforming, and organizing raw data** before analysis.

It ensures that data is **accurate, complete, and formatted correctly** for further processing.

Why is Data Preparation Important?

- **Ensures Data Quality** – Removes missing values, duplicates, and errors.
- **Improves Model Accuracy** – Clean data leads to **better predictions** in machine learning.
- **Speeds Up Analysis** – Well-prepared data makes analysis **faster and more efficient**.

Steps in Data Preparation

1. **Data Collection** – Import data from **CSV, Excel, databases, APIs**.
2. **Handling Missing Values** – Fill or remove missing data.
3. **Removing Duplicates** – Avoid redundant or repeated entries.
4. **Correcting Data Types** – Convert text, numbers, and dates into the correct format.
5. **Feature Scaling** – Normalize values for better comparisons.
6. **Encoding Categorical Data** – Convert non-numerical data into numerical format.

Real-Life Applications of Data Preparation

- **E-commerce** – Cleaning customer data to improve recommendations.
- **Healthcare** – Preparing patient records for accurate diagnosis.
- **Finance** – Filtering transaction data for fraud detection.

Anecdote: How Data Cleaning Helped a Business Save Millions

A retail company faced issues with **wrong product pricing** due to duplicate and inconsistent data.

By using **data preparation techniques**, they corrected errors, which **saved them millions in lost revenue**.

Section 2: Practice

1. Importing and Inspecting Data

```
import pandas as pd


# Reading a CSV file
df = pd.read_csv("data.csv")


# Display first five rows
print(df.head())


# Checking data types and missing values
print(df.info())
```

---

2. Handling Missing Values

```
# Checking for missing values
print(df.isnull().sum())

# Filling missing values with mean
df["Age"].fillna(df["Age"].mean(), inplace=True)

# Dropping rows with missing values
df.dropna(inplace=True)
```

---

3. Removing Duplicates

```
# Checking for duplicate rows
print(df.duplicated().sum())

# Removing duplicates
df.drop_duplicates(inplace=True)
```

---

4. Converting Data Types

```
# Converting date column to datetime format
df["Date"] = pd.to_datetime(df["Date"])

# Converting object (string) to numeric
df["Salary"] = pd.to_numeric(df["Salary"], errors="coerce")
```

---

5. Feature Scaling (Normalization)

```
from sklearn.preprocessing import MinMaxScaler


# Applying Min-Max Scaling to normalize values between 0 and 1
scaler = MinMaxScaler()
df[["Salary"]] = scaler.fit_transform(df[["Salary"]])
```

---

6. Encoding Categorical Variables

```
# Converting categorical data into numerical format using one-hot encoding
df = pd.get_dummies(df, columns=["Gender", "City"])
print(df.head())
```

Section 3: Know More

Frequently Asked Questions (FAQs)

1. Why do we need to handle missing values?

Missing values can cause **errors in analysis** and lead to **biased predictions**.

2. How do I detect outliers in my data?

Use df.describe() to check for extreme values, or apply visualization methods like box plots.

3. What happens if I don't remove duplicates?

Duplicate records can **distort statistics** and **mislead analysis**.

4. How does feature scaling improve data analysis?

Scaling ensures that **different units of measurement do not affect comparisons**, especially in machine learning.

5. What is the best way to encode categorical data?

- **Label Encoding** – Assigns numbers (e.g., Male=0, Female=1).
- **One-Hot Encoding** – Creates separate columns for each category (e.g., City_A, City_B).

Conclusion:

Data preparation is the **foundation of accurate and efficient analysis**.

By **cleaning, transforming, and structuring data**, we can extract **valuable insights and make informed decisions**.